

Reviewer Pack — Minimal Eta-Invariant Correlation with Boolean Circuit Monot...

Entry 2beb47590b31 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 15:51:33 UTC

Minimal Eta-Invariant Correlation with Boolean Circuit Monotone Width

Entry ID: 2beb47590b31 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 15:51:33 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Eta Invariant) **Field B** (complexity object): Boolean Circuit Complexity

Statement:

For every d -regular graph G , the minimal eta-invariant of its associated complex $K(G)$ is linearly correlated with its monotone width $w_{\text{mon}}(G)$, such that $\eta_{\text{min}}(K(G)) = \Theta(w_{\text{mon}}(G))$.

Rationale (proposer's reasoning):

The eta-invariant is an invariant in algebraic topology that measures the complexity of a space. By relating it to circuit monotone width, we may uncover a connection between topological complexity and boolean circuit complexity, potentially leading to insights into circuit lower bounds.

Taxonomy category: resolution-proof-complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0810cca81b2fd178

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the conjecture that minimal eta-invariant of complex $K(G)$ is linearly correlated with monotone width $w_{\text{mon}}(G)$, support is indicated by a Pearson correlation coefficient ≥ 0.7 for at least 3 out of 5 seeds, and the mean absolute difference between $\eta_{\text{min}}(K(G))$ and $\Theta(w_{\text{mon}}(G)) \leq 2$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 1 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Eta Invariant" AND "Boolean Circuit Monotone Width" - "Minimal Eta-Invariant" AND d-regular graph $K(G)$ $w_{\text{mon}}(G)$ - "correlation" minimal eta-invariant monotone width $\theta(w_{\text{mon}}(G))$

Top relevant hits considered: - [s2:10.1007/978-3-030-58150-3_40] Succinct Monotone Circuit Certification: Planarity and Parameterized Complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction

def generate_d_regular_graph(n, d):
    if n % d != 0:
        raise ValueError("Graph size must be a multiple of the degree")

    graph = [[] for _ in range(n)]
    edges_added = set()

    def add_edge(u, v):
        if (u, v) not in edges_added and (v, u) not in edges_added:
            graph[u].append(v)
            graph[v].append(u)
            edges_added.add((u, v))
            edges_added.add((v, u))

    for i in range(n):
        for j in range(i + 1, n):
            if len(graph[i]) < d and len(graph[j]) < d:
                add_edge(i, j)

    return graph

def compute_eta_invariant(graph):
    # Placeholder implementation of eta-invariant computation
    # This is a dummy function and should be replaced with the actual algorithm
    return Fraction(0, 1)

def measure_monotone_width(graph):
    n = len(graph)
```

```

max_width = 0

def dfs(node, visited, path):
    nonlocal max_width
    if len(path) > max_width:
        max_width = len(path)

    for neighbor in graph[node]:
        if neighbor not in visited:
            visited.add(neighbor)
            dfs(neighbor, visited, path + [neighbor])
            visited.remove(neighbor)

for i in range(n):
    visited = set([i])
    dfs(i, visited, [i])

return max_width

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    eta_values = []
    width_values = []

    for n in n_values:
        graph = generate_d_regular_graph(n, 2) # Example degree
        eta = compute_eta_invariant(graph)
        width = measure_monotone_width(graph)

        eta_values.append(eta)
        width_values.append(width)

    correlation_coefficient = sum((a - b) * (c - d) for a, b, c, d in zip(eta_values, [sum(eta_val
    mean_abs_difference = sum(abs(a - b) for a, b in zip(eta_values, [Fraction(n, 1) for n in width

    return {
        "metric_name": "Correlation Coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(n_values),
        "n_max": max(n_values),
        "conjecture_holds": correlation_coefficient >= 0.7 and mean_abs_difference <= 2,
        "counterexample": "" if correlation_coefficient >= 0.7 and mean_abs_difference <= 2 else '
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [3, 5, 7, 11, 13, 17, 19, 23, 29,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...{result}...}}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)

```

```

std_value = (sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results)) ** 0.5
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next((seed for seed, r in zip(seeds, results) if not r["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f61e7f5.py", line 97, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f61e7f5.py", line 72, in run_trial
    graph = generate_d_regular_graph(n, 2) # Example degree
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f61e7f5.py", line 20, in generate_d_regular_graph
    raise ValueError("Graph size must be a multiple of the degree")
ValueError: Graph size must be a multiple of the degree

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the Pearson correlation coefficient and mean absolute difference as required by the support condition. | next: Investigate the cause of the crash in the test code and attempt to run the test again with appropriate input parameters.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12276
2	preregistration	ollama_remote	glm4:latest	0	0	9714
3	novelty	ollama_remote	glm4:latest	0	0	11211

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	9875
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15018
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14824
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14006
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12352
9	judge	ollama_remote	glm4:latest	0	0	12987

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 112263 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/2beb47590b31.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/2beb47590b31.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/2beb47590b31.tar.gz* (if generated)